

h2kvmctl - Primary CLI Command Guide

Modern, concise CLI for interactive Hyper2KVM workflows following the kubectl naming pattern

Overview

`h2kvmctl` (Hyper2KVM Control) is the primary command for **interactive CLI and command-line usage**, designed to provide a modern, concise interface following established patterns from kubectl, helmctl, and similar tools.

Command Purpose Distinction

Hyper2KVM provides two commands serving complementary purposes:

- **`h2kvmctl`** - Primary for **interactive CLI workflows**
 - Command-line migrations and scripting
 - Interactive operations and testing
 - Developer-friendly for quick tasks
 - Follows kubectl-style naming pattern
- **`hyper2kvm`** - Primary for **daemon mode and services**
 - Systemd service units (`hyper2kvm daemon`)
 - Background processing and automation
 - Traditional daemon naming convention
 - Better suited for service contexts

Neither is deprecated - both are actively maintained and serve different use cases.

Why h2kvmctl for CLI?

- **Shorter:** 4 characters less than `hyper2kvm` (saves typing)
- **Modern:** Follows industry-standard `*ctl` naming pattern
- **Memorable:** Easy to remember (`h2kvm` + `ctl`)

- **Professional:** Aligns with Kubernetes ecosystem conventions
 - **CLI-Focused:** Clear signal this is for command-line usage
-

Installation

Both `h2kvmctl` and `hyper2kvm` are installed together:

```
# Install via pip
pip install hyper2kvm

# Both commands are now available
h2kvmctl --version      # For interactive CLI usage (recommended)
hyper2kvm --version     # For daemon mode and services
```

Command Equivalence

Important: `h2kvmctl` and `hyper2kvm` are **functionally identical** - they call the same Python function with zero performance difference.

```
# These work exactly the same for CLI usage
h2kvmctl --config migration.yaml
hyper2kvm --config migration.yaml

# Same for all operations
h2kvmctl --cmd local --vmdk /vms/vm.vmdk
hyper2kvm --cmd local --vmdk /vms/vm.vmdk

# Same for TUI
h2kvmctl-tui
hyper2kvm-tui
```

When to use which: - **Interactive migrations:** `h2kvmctl --config migration.yaml` (shorter, clearer) - **Daemon mode:** `hyper2kvm daemon --config daemon.yaml` (traditional daemon naming) - **Systemd services:** Use `hyper2kvm` in ExecStart (daemon convention)

Quick Start with h2kvmctl

1. Basic Migration

```
# Create config
cat > migration.yaml <<EOF
command: local
vmdk: /vms/centos7.vmdk
output_dir: ./out
to_output: centos7.qcow2
out_format: qcow2
compress: true
fstab_mode: stabilize-all
regen_initramfs: true
EOF

# Run migration
sudo h2kvmctl --config migration.yaml
```

2. Remote ESXi Fetch

```
# Fetch and convert from ESXi
sudo h2kvmctl --config fetch.yaml
```

fetch.yaml:

```
command: fetch-and-fix
host: 192.168.1.100
user: root
identity: ~/.ssh/id_rsa
remote: /vmfs/volumes/datastore1/vm/vm.vmdk
output_dir: ./migrated
to_output: vm.qcow2
```

3. Live Fix (SSH)

```
# Fix running VM without downtime
sudo h2kvmctl --config live-fix.yaml
```

live-fix.yaml:

```
command: live-fix
host: production-vm.example.com
user: root
identity: ~/.ssh/id_rsa
fstab_mode: stabilize-all
regen_initramfs: true
```

4. Batch Migration

```
# Migrate multiple VMs in parallel
sudo h2kvmctl --config batch.yaml
```

batch.yaml:

```
command: local
batch_manifest: vms.json
batch_parallel: 3
batch_continue_on_error: true
output_dir: ./batch-output
```

Command Reference

All commands work identically with `h2kvmctl` :

Supported Commands

Command	Description	Example
<code>local</code>	Convert local VMDK	<code>h2kvmctl --cmd local --vmdk vm.vmdk</code>
<code>migrate</code>	Alias for <code>local</code>	<code>h2kvmctl --cmd migrate --vmdk vm.vmdk</code>
<code>fetch-and-fix</code>	Remote ESXi fetch	<code>h2kvmctl --config fetch.yaml</code>
<code>ova</code>	Extract OVA	

Command	Description	Example
		<code>h2kvmctl --cmd ova --ova vm.ova</code>
<code>ovf</code>	Extract OVF	<code>h2kvmctl --cmd ovf --ovf vm.ovf</code>
<code>vhd</code>	Convert VHD	<code>h2kvmctl --cmd vhd --vhd vm.vhd</code>
<code>ami</code>	Extract AMI tarball	<code>h2kvmctl --cmd ami --ami ami.tar.gz</code>
<code>raw</code>	Convert raw image	<code>h2kvmctl --cmd raw --raw disk.img</code>
<code>live-fix</code>	SSH-based fixing	<code>h2kvmctl --config live.yaml</code>
<code>libvirt-xml</code>	Parse libvirt XML	<code>h2kvmctl --cmd libvirt-xml --xml vm.xml</code>
<code>vsphere</code>	vSphere operations	<code>h2kvmctl --config vsphere.yaml</code>
<code>azure</code>	Azure migration	<code>h2kvmctl --config azure.yaml</code>
<code>daemon</code>	Watch directory	<code>h2kvmctl --config daemon.yaml</code>
<code>generate-systemd</code>	Generate unit file	<code>h2kvmctl --cmd generate-systemd</code>

TUI Dashboard

Launch the Terminal User Interface:

```
# Primary command (recommended)
h2kvmctl-tui
```

```
# Legacy command (still works)
hyper2kvm-tui
```

The TUI provides: - Real-time migration progress - System resource monitoring - Live log streaming - Interactive controls

Backwards Compatibility

No Migration Required

Existing automation continues to work - both commands coexist:

```
# Old scripts using hyper2kvm - still work!
hyper2kvm --config migration.yaml # Works

# New scripts using h2kvmctl - recommended
h2kvmctl --config migration.yaml # Works
```

Gradual Adoption

1. **Existing CI/CD:** Keep using `hyper2kvm` (no changes needed)
 2. **New Projects:** Start with `h2kvmctl` (modern, concise)
 3. **Documentation:** Gradually update examples to `h2kvmctl`
 4. **No Deprecation:** Both commands maintained indefinitely
-

Common Workflows

Standard Migration Workflow

```
# 1. Create config
vim migration.yaml

# 2. Run migration
sudo h2kvmctl --config migration.yaml

# 3. Verify output
ls -lh ./out/
```

```
# 4. Import to libvirt
virsh define ./out/vm.xml
virsh start vm
```

Batch Processing Workflow

```
# 1. Create batch manifest
cat > batch.json <<EOF
{
  "migrations": [
    {"vmdk": "/vms/vm1.vmdk", "to_output": "vm1.qcow2"},
    {"vmdk": "/vms/vm2.vmdk", "to_output": "vm2.qcow2"},
    {"vmdk": "/vms/vm3.vmdk", "to_output": "vm3.qcow2"}
  ]
}
EOF

# 2. Create batch config
cat > batch.yaml <<EOF
command: local
batch_manifest: batch.json
batch_parallel: 3
output_dir: ./batch-out
EOF

# 3. Run batch migration
sudo h2kvmctl --config batch.yaml
```

Remote Fetch Workflow

```
# 1. Setup SSH key access to ESXi
ssh-copy-id root@esxi-host

# 2. Create fetch config
cat > fetch.yaml <<EOF
command: fetch-and-fix
host: esxi-host
user: root
remote: /vmfs/volumes/datastore1/vm/vm.vmdk
output_dir: ./fetched
to_output: vm.qcow2
fstab_mode: stabilize-all
regen_initramfs: true
EOF
```

```
# 3. Fetch and convert
sudo h2kvmctl --config fetch.yaml
```

Best Practices

Recommended

1. **Use h2kvmctl for CLI work:** Shorter, modern, perfect for interactive use
2. **Use hyper2kvm for daemon mode:** Traditional naming for background services
3. **YAML configs over CLI flags:** More maintainable
4. **Enable compression:** Saves disk space
5. **Use stabilize-all for fstab:** Prevents mount failures
6. **Test with libvirt_test:** Verify before production

Cautions

1. **Choose the right command:** `h2kvmctl` for CLI, `hyper2kvm` for daemons (but both work anywhere)
2. **Don't skip backups:** Use `no_backup` carefully
3. **Don't force-push broken configs:** Test locally first

Examples

Example 1: Simple VMDK Conversion

```
h2kvmctl --cmd local \  
  --vmdk /vms/centos7.vmdk \  
  --output-dir ./out \  
  --to-output centos7.qcow2 \  
  --compress
```

Example 2: Windows VM with VirtIO

```
command: local
vmdk: /vms/windows-server-2019.vmdk
output_dir: ./out
to_output: windows-server.qcow2
out_format: qcow2

# Windows-specific
windows: true
virtio_drivers_dir: /usr/share/virtio-win

# Fixes
fstab_mode: stabilize-all
compress: true
libvirt_test: true
```

```
sudo h2kvmctl --config windows.yaml
```

Example 3: Dry Run Preview

```
# Preview without making changes
sudo h2kvmctl --config migration.yaml --dry-run -vv
```

Example 4: Generate Systemd Unit

```
# Create systemd service for daemon mode
sudo h2kvmctl --cmd generate-systemd \
  --systemd-output /etc/systemd/system/hyper2kvm-daemon.service
```

Help and Documentation

Get Help

```
# Show help
h2kvmctl --help

# Show version
```

```
h2kvmctl --version

# Dump config (debug)
h2kvmctl --config migration.yaml --dump-config

# Dump parsed args (debug)
h2kvmctl --config migration.yaml --dump-args
```

Documentation Links

- CLI Reference: <docs/guides/cli/reference.md>
- Beginner Tutorial: <docs/tutorials/01-beginner-migration.md>
- Main Index: <docs/index.md>

Comparison: h2kvmctl vs hyper2kvm

Feature	h2kvmctl	hyper2kvm
Primary Use	Interactive CLI work	Daemon mode & services
Length	8 chars	12 chars
Performance	Identical	Identical
Functionality	100% same	100% same
Recommended for	Command-line migrations	Systemd services, daemons
Deprecation	Never	Never
Pattern	kubectl-style	Traditional daemon naming
Status	Active	Active

Verdict: - Use `h2kvmctl` for interactive CLI work (shorter, modern) - Use `hyper2kvm` for daemon mode and systemd services (traditional daemon naming) - Neither is deprecated - both actively maintained for their respective purposes

Troubleshooting

Command Not Found

```
# After pip install, if command not found:
pip show hyper2kvm          # Verify installation
which h2kvmctl              # Check if in PATH

# Development mode:
./h2kvmctl --version        # Use local wrapper
```

Permission Denied

```
# Most operations need root/sudo
sudo h2kvmctl --config migration.yaml

# Check permissions
ls -l /vms/vm.vmdk
```

Config Not Found

```
# Use absolute path
sudo h2kvmctl --config /full/path/to/migration.yaml

# Or relative to current directory
sudo h2kvmctl --config ./migration.yaml
```

Next Steps

1. **Read CLI Reference:** Complete command documentation
 2. **Try Examples:** Run sample migrations in `/examples/yaml/`
 3. **Explore Features:** Test batch, live-fix, vsphere modes
 4. **Join Community:** Report issues, contribute improvements
-

Last Updated: January 2026 **Command Status:** Production Ready **Supported Versions:**
Python 3.10+, All platforms